The Experience Machine: A Conceptual Computation Model for Experience-Driven Intelligence

Chengbo Yuan

Tsinghua University
ycb24@mails.tsinghua.edu.cn

Abstract: Humans learn from experience, while most current embodied agents do not. Modern robots are largely trained from offline demonstrations and exhibit only limited adaptation after deployment, often within specific domains. We argue that intelligence should continuously adapt to experience. We introduce the Experience Machine (EM), a conceptual model for this setting as a computational paradigm for experience-driven intelligence. EM draws inspiration from the computational abstraction of the Turing Machine and views experience as a paper-tape written with state, intent, action, and consequence critique in temporal order. The machine continuously reads the tape and performs processing and prediction, unifying policy inference, world modeling, and outcome evaluation, while updating its own internal belief. The key idea is that the context written on the tape can come from diverse sources: internal imagination, real-world interaction, and human supervision. We demonstrate that this provides a natural path toward embodied reasoning, self-exploration, reinforcement learning from environment, and human-machine interaction. Rather than treating these capabilities as isolated research problems, EM places all of them within a single computational framework. It also naturally supports adaptive test-time computation, allocating more computation to complex or uncertain situations while using less for simple ones. Rather than proposing specific empirical results, this paper aims to introduce a unified computational framework that connects a growing set of directions and capabilities toward next-generation intelligence. We envision EM as a new paradigm for experience-driven intelligence and hope it inspires concrete instantiations in the future. https://michaelyuancb.github.io/experience-machine-concept

Keywords: Experience-Driven Intelligence, Computational Model

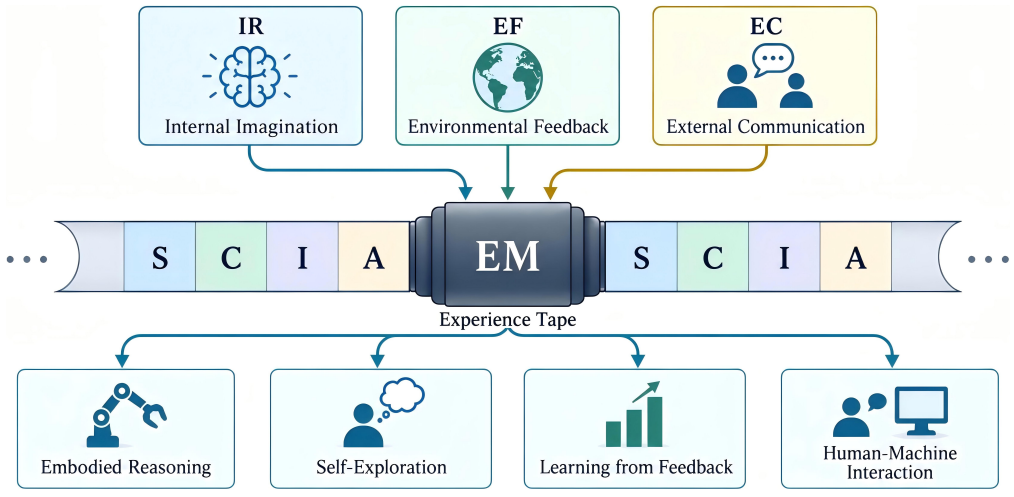

Figure 1: Overview of the Experience Machine. Different experience sources write state, critique, intent, and action sequentially onto a shared Experience Tape, enabling diverse experience-driven capabilities through the same underlying computation.

1 Introduction

Experience is the teacher of all things. — Julius Caesar

Humans become intelligent not only by acquiring knowledge, but by continuously computing and changing through experience. Before acting, they imagine possible futures; after acting, they observe real-world consequences, reflect on the outcomes, and incorporate feedback from others and the environment. Modern embodied agents are fundamentally different. Most are trained offline, aligned for specific domains or tasks, and then deployed as largely fixed systems. They may accumulate interaction histories, but those experiences rarely become a persistent substrate that reshapes how the agent reasons, predicts, and acts.

Progress has been made, but it remains fragmented and far from sufficient. Current reinforcement learning methods learn from real-world trial and error, but are often limited to task-specific optimization [1, 2]. In-context learning adapts behavior from recent experience or human demonstrations [3, 4], while test-time training [5, 6] and world models [7, 8] address adaptation and future prediction through separate mechanisms. However, these works still focus on adding experience-related capabilities to existing embodied frameworks, rather than defining a new paradigm built around experience itself. This raises a natural question: can we build a unified computational model in which all of these capabilities emerge naturally from experience?

In this paper, we introduce the **Experience Machine (EM)** as a conceptual framework for experience-driven intelligence. EM provides a computational abstraction for a family of systems that continuously process experience to support reasoning, prediction, action, self-improvement, and human-machine interaction. Instead of prescribing a specific architecture, EM defines a unified computational form in which these capabilities emerge from **the same underlying process** of experience accumulation and computation while allowing diverse concrete instantiations.

The Experience Machine draws inspiration from the computational abstraction of the Turing Machine (TM) [9]. Inspired by the paper-tape formulation of the TM, EM treats accumulated experience as an evolving **Experience Tape**, where states, critiques, intents and actions are written in temporal order. The machine continuously reads this tape to make predictions and decisions, repeatedly performing policy inference, world modeling, outcome evaluation and new intent assignment. **Our central observation is that seemingly different experience-driven capabilities can arise from the same computational process simply by changing who or what writes to the Experience Tape.** In this way, EM unifies experience-related capabilities such as embodied reasoning, self-exploration, reinforcement learning from environmental feedback, in-context learning, and human-machine interaction within a simple computational framework.

To demonstrate the Experience Machine, we first classify experience into three major types: internal imagination, environmental feedback, and external communication. We also formulate EM from both a Turing Machine perspective and a mathematical perspective, and introduce the inference loop during deployment. Building on this formulation, we systematically show how a range of emerging experience-driven capabilities can be naturally derived within the unified EM framework. Finally, we revisit representative prior works such as Decision Transformer [10], RoboTTT [5], DreamerV4 [11], Meta-RL [12], RLVR agents [13] and Voyager [14] and show how different components of their computation can be interpreted through the EM abstraction. We hope this paper provides a new unified perspective on experience-driven intelligence and serves as a conceptual foundation for ongoing efforts to instantiate EM in concrete learning systems.

2 The Experience Machine

2.1 The Experience Tape

The Experience Machine draws inspiration from the tape-based computational abstraction of the Turing Machine [9]. It conducts computation over a continuously evolving paper tape, which we

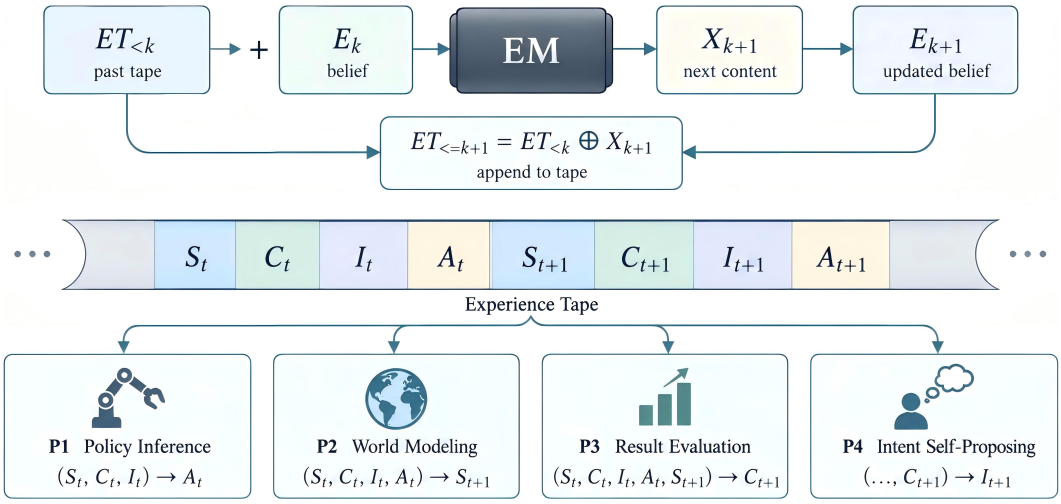

Figure 2: Next-content prediction over the Experience Tape. EM recurrently predicts the next tape content and updates its internal belief, unifying policy inference, world modeling, result evaluation, and intent self-proposing.

refer to as the **Experience Tape** (ET). Rather than storing abstract symbols, the Experience Tape records the interaction history of an intelligent system in temporal order.

Formally, let S_t, C_t, I_t, and A_t denote the representation of state, critique, intent, and action at time t, respectively. Here, S_t represents the state of world observation, C_t evaluates the accumulated experience, I_t specifies the objective or demand for the next interaction, and A_t denotes the action produced by policy inference. The Experience Tape is then represented as a temporally evolving sequence:

$$ET = (\ldots, S_t, C_t, I_t, A_t, S_{t+1}, C_{t+1}, I_{t+1}, A_{t+1}, \ldots). \tag{1}$$

The key idea of the Experience Machine is to treat the Experience Tape not as a static offline trajectory, but as **a dynamic computational object whose content can be written by different sources**. Based on who or what writes to the tape, we classify experience into three major types:

E1: Imagination Reasoning (IR). Experience generated internally by the model is treated as imagined experience. It is the key component to support embodied reasoning for complex tasks, self-exploration, and learning from failure.

E2: Environmental Feedback (EF). When the tape is written by the physical environment after executing action A_t, it records grounded interaction experience. This corresponds to the most common form of experience considered in existing learning systems.

E3: External Communication (EC). Experience can also be written through communication with external agents, especially humans. Human instructions may specify I_t, demonstrations may provide action sequences, and corrections or evaluations may contribute to C_t. This supports task assignment, policy guidance, in-context adaptation, and human-machine interaction.

In Section 3, we further analyze how different combinations of them can give rise to a broad range of experience-driven intelligent behaviors. Next, we describe how computation is carried out over the Experience Tape.

2.2 Computational Model: Next-Content Prediction

Given the Experience Tape (ET), the Experience Machine (Figure 2) performs computation through sequential "Next-Content Prediction". We distinguish the index t from the tape index k: t denotes

Algorithm 1 A Generic Experience Machine Inference Loop

Initialize $ET_{\leq 0}$ and belief E_0
We omit the update of belief E_k for clarity.
for $k = 0, 1, 2, \ldots$ **do**
  Determine the next content type $X_k \in \{S, C, I, A\}$
  if $X_k \in \{C, I\}$ **then**
    $X_k \leftarrow \text{LatestAvailable}\{\text{Human(EC)} \succ \text{Environment(EF)} \succ \text{Model(IR)}\}$
  else if $X_k = A$ **then**
    $X_k \leftarrow \text{Model}(ET_{<k})$
    if $X_{k-1} = I$ is model-generated and indicates execution **then**
      $\text{Execute}(X_k)$
    end if
  else if $X_k = S$ **then**
    if $X_{k-1} = A$ was executed **then**
      $X_k \leftarrow \text{Environment}$
    else
      $X_k \leftarrow \text{Model}(ET_{<k})$
    end if
  end if
  $ET_{\leq k} \leftarrow ET_{<k} \oplus X_k$
end for

the experience timestamp, while k denotes the position of a content item on the Tape. Let

$$X_k \in \{S_t, C_t, I_t, A_t\}$$

denote the k-th content item on the tape, and let E_k denote the internal belief of the machine after processing X_k, which continuously evolves with the accumulated experience history. The Experience Machine (EM) then performs the recurrent computation

$$(X_{k+1}, E_{k+1}) = EM(ET_{\leq k}, E_k), \tag{2}$$

where the predicted X_{k+1} is written back to the tape and becomes part of the context for subsequent computation. Following the tape order, EM sequentially predicts $S_t \rightarrow C_t \rightarrow I_t \rightarrow A_t \rightarrow S_{t+1}$. Specifically, EM contains four prediction functions. For clarity, we omit the annotation of E_{k+1} in the following formulations:

P1: Policy Inference. $(S_t, C_t, I_t, E_k, ET_{\leq k}) \rightarrow A_t$.

P2: World Modeling. $(S_t, C_t, I_t, A_t, E_k, ET_{\leq k}) \rightarrow S_{t+1}$.

P3: Result Evaluation. $(S_t, C_t, I_t, A_t, S_{t+1}, E_k, ET_{\leq k}) \rightarrow C_{t+1}$.

P4: Intent Self-Proposing. $(S_t, C_t, I_t, A_t, S_{t+1}, C_{t+1}, E_k, ET_{\leq k}) \rightarrow I_{t+1}$. The model reflects on previous experience and proposes a new intent for task completion or self-exploration. We use intent in a broad sense: beyond specifying an external task objective, a model-generated intent may also represent an intermediate decision to continue internal reasoning, execute an action, or propose a new subgoal, as detailed in the next section.

These are not four independent computational paradigms, but four instances of predicting the next content on the same evolving tape. These functionalities have been studied independently in prior work [15, 8, 16, 17], providing useful building blocks for future instantiations of the Experience Machine. Although EM may support all four types of prediction, the corresponding content on the Experience Tape does not always need to be generated by the model itself; it may instead be written by external sources such as the physical environment or human supervisors. Therefore, each prediction function is activated only when required by the current experience-driven behavior. We also note that the Experience Machine need not take the form of a single unified end-to-end model. It may instead be instantiated as a collection of coordinated models jointly trained over a shared representation space.

2.3 Adaptive Experience Inference Loop

During deployment, the Experience Machine continuously fills the Experience Tape according to the availability and source of each content item. A generic inference protocol is summarized in Algorithm 1. For critique C_t and intent I_t, externally grounded signals take precedence over model-generated predictions, following the priority $\mathrm{Human}(EC) \succ \mathrm{Environment}(EF) \succ \mathrm{Model}(IR)$, where EC, EF, and IR denote the three experience types introduced in Section 2.1. This priority ensures that the machine responds first to external stimuli and demands before relying on its own reasoning or imagination, which is important for safe interaction.

Actions are always generated by the model, but are executed in the physical environment only when the preceding model-generated intent requests execution. When I_t is provided externally, it specifies the target or demand. When I_t is generated by the model itself, it drives internal reasoning toward task completion or self-exploration and determines what should be done next. It also decides whether the generated action should be executed based on the accumulated experience and the estimated task complexity. Accordingly, the subsequent state is written either by the environment after real execution or by the model during internal imagination. In this way, real interaction and internal reasoning share the same sequential computation over the Experience Tape.

3 One Computational Model for All Experience-Driven Capabilities

In this section, we show how different experience-driven capabilities can emerge from the same Experience Machine computation by changing the sources of S, C, I, and A on the Experience Tape (Figure 3). We first analyze several representative capability modes individually, and then demonstrate how they can be dynamically combined within a single evolving computation process.

3.1 Adaptive Embodied Reasoning ($\{S_{t+1}, C_{t+1}, I_{t+1}\} \in IR$)

The Experience Machine performs adaptive embodied reasoning through iterative imagination and self-evaluation over the Experience Tape. Consider a fixed intent I_t specified by the environment or an external supervisor. Before execution, the model proposes an action A_t, predicts its possible consequence S_{t+1}, and evaluates the imagined outcome through C_{t+1}. Based on the accumulated experience and the current critique, the model then determines whether the proposed action is sufficiently aligned with the target (I_{t+1}), as shown in Line 9 of Algorithm 1.

The action is executed only when the model has sufficient confidence that it can advance the assigned intent. Otherwise, the imagined actions are written back to the Experience Tape, and the machine continues reasoning from the updated context. This naturally results in adaptive test-time computation: complex or uncertain situations induce longer chains of imagination and evaluation, whereas familiar or simple situations can lead directly to action with little or no additional reasoning. In this sense, **EM allocates test-time computation according to task difficulty**, resembling human or Large-Language Models (LLMs) [13] reasoning that may involve detailed internal simulation for difficult decisions but only coarse prediction or direct action for simple ones.

3.2 Self Exploration ($\{S_{t+1}\} \in EF$, $\{C_{t+1}, I_{t+1}\} \in IR$)

After executing action A_t, the model receives the resulting state S_{t+1} from the real world, as shown in Line 14 of Algorithm 1. It then generates a critique C_{t+1} to evaluate the outcome and, based on this evaluation and the accumulated experience, proposes the intent I_{t+1} for subsequent interaction. This process enables self-exploration in the Experience Machine, allowing the model to decide what to do next under the broader intent encoded in its history.

If the broader intent corresponds to a specific task, the model continues reasoning toward task completion as described above. If the intent is underspecified, the model may instead self-propose a new target [17] and interact with the real world to acquire new experience. Throughout this process, the belief E_k is updated to integrate past experience into the machine's internal knowledge.

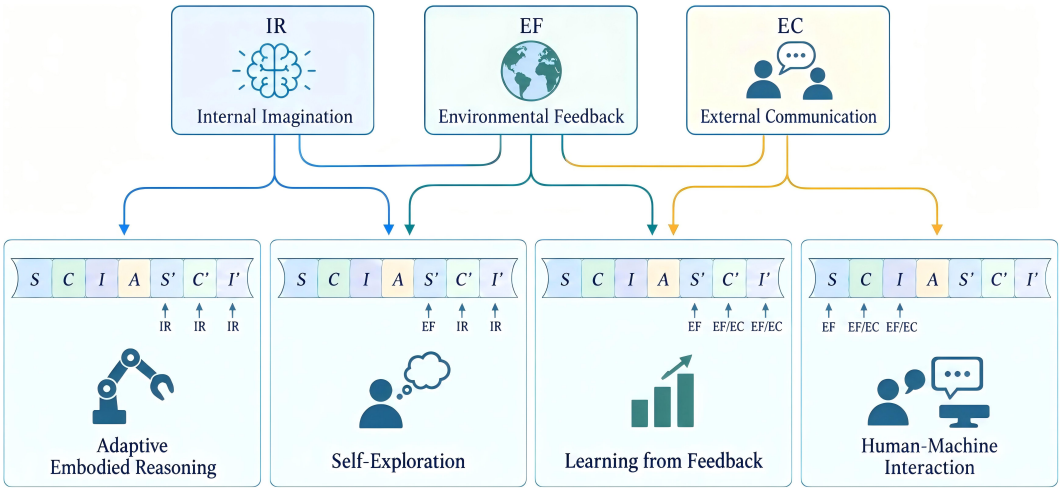

Figure 3: Different experience-source assignments over the same tape give rise to different capabilities, including embodied reasoning, self-exploration, learning from feedback, and human-machine interaction.

3.3 Learning from Feedback ($\{S_{t+1}\} \in EF$, $\{C_{t+1}, I_{t+1}\} \in EF/EC$)

The Experience Machine does not rely solely on model-generated critiques and intents. After an action is executed, the resulting experience can also be evaluated by the environment (EF) or external supervisors (EC), which may directly provide C_{t+1} or further specify I_{t+1}. These externally provided contents are written onto the Experience Tape and become part of the context for subsequent prediction and action.

Such feedback serves two roles. From an inference perspective, it directly steers the current interaction by correcting the model's evaluation or changing the next intent. From a learning perspective, the externally grounded C_{t+1} and I_{t+1} [18] can supervise the model's own prediction of critiques and intents. Therefore, feedback not only changes what the machine does next, but also improves how it evaluates and plans in future interactions.

3.4 Human-Machine Interaction ($\{S_t\} \in EF$, $\{C_t, I_t\} \in EF/EC$)

Learning from feedback describes how the external world influences EM by evaluating previous experience through C_{t+1} or modifying the subsequent intent I_{t+1}. Human-machine interaction provides a broader interface: the environment or external agents can also directly write S_t, C_t and I_t onto the Experience Tape before or during interaction. In this way, external agents can not only evaluate what the machine has done, but also provide new observations, assign tasks, modify goals, and guide what the machine should do next. These written contents immediately become part of the Experience Tape and therefore change the context used for subsequent prediction and action.

Because S_t, C_t, and I_t are all processed within the same evolving tape, task assignment, guidance, and feedback do not require separate interaction mechanisms. Under the Experience Machine framework, feedback and intent can be viewed, to some extent, as a dual pair: feedback evaluates behavior from past outcomes, while intent specifies desired future behavior. Despite operating in opposite temporal directions, both steer the machine through the same underlying computation over the Experience Tape.

3.5 Adaptive Experience-Driven Intelligence

Finally, we show that by dynamically combining state S, critique C, intent I, and action A from internal imagination (IR), environmental feedback (EF), and external communication (EC), the Experience Machine can exhibit adaptive experience-driven behaviors. Rather than explicitly selecting

a predefined capability mode, EM naturally transitions between different behaviors according to who or what writes the current contents of the Experience Tape.

For example, after executing A_t in the real world, the model may generate its own critique C_{t+1} based on the observed outcome. Before it proposes the next intent, however, a human may write a new I_{t+1} onto the Experience Tape. The subsequent action is then predicted from both the model's self-evaluation of previous experience and the newly assigned external intent.

Similarly, during internal embodied reasoning, the model may be generating an imagined action sequence. Even without observing the full internal imagination chain, a human can still provide a critique based on the current real-world situation. Once written onto the Experience Tape, this external critique becomes part of the context for subsequent imagined prediction and can redirect the ongoing reasoning process.

In conclusion, EM does not treat embodied reasoning, feedback learning, self-exploration, and human-machine interaction as isolated modes. Instead, these behaviors emerge dynamically from different combinations of experience sources within the same evolving computational process.

4 Connections to Existing Experience-Driven Research

We discuss how several closely related research directions connect to the Experience Machine framework. In particular, we focus on four areas: sequential decision modeling, history-conditioned adaptation, reinforcement learning and LLM-based instantiation.

4.1 Sequential Decision Modeling

Prior work has explored modeling experience as sequences of states, actions, and evaluation signals. Decision Transformer [10] predicts actions conditioned on trajectory history, while Trajectory Transformer [19] further autoregressively models future trajectory contents, including states, actions, and rewards. More recent works extend this sequential modeling paradigm to larger foundation-model settings, such as Gato [20], DreamZero [21] Lingbot-VA [22] and Motus2 [23].

The Experience Machine extends these methods by explicitly introducing critique and intent and, more importantly, by allowing different experience sources to dynamically write different contents onto the sequence, thereby giving rise to different experience-driven capabilities. A key training challenge for EM is therefore how to enable the model to handle contents written from heterogeneous sources on the Experience Tape. This may require richer interaction data or learning objectives that explicitly expose the model to different experience sources [24]. Nevertheless, these works provide useful preliminary explorations of how an Experience Machine could be pretrained from offline experience data to acquire the causal and conditional dependencies required for computation over the Experience Tape.

4.2 History-Conditioned Adaptation

A key feature of the Experience Machine is that its behavior adapts to accumulated experience during deployment. A central challenge is therefore how to efficiently compress and retain history so that it can influence future actions. Recent works such as MEM [25], Hamlet [26], and RoboTTT [5] explore different forms of experience compression through language, latent tokens, and fast weights. A broader line of work, including RoboTTT [5], Reflective VLA [27], IC-WM [28], LocoFormer [29], GEN-1.5 [4], Skild-S1 [30], further formulates adaptation in an in-context learning setting [3]. These studies show that online adaptation from recent experience can improve generalization, failure recovery, and intent assignment from human demonstrations.

From the EM perspective, these works provide important foundations for experience compression, memory retention, and history-conditioned adaptation, demonstrating that accumulated experience can effectively influence future behavior during deployment. Building on these capabilities, a further open problem for EM is how to sustain adaptation over much longer interaction horizons. As experience continues to accumulate, explicit context or external memory alone may become insuffi-

cient, motivating the need for continual learning [31] to progressively consolidate useful experience into the model itself [32] while preserving fast adaptation from recent context.

4.3 Reinforcement Learning

Learning from experience has also been extensively formulated as a reinforcement learning (RL) problem. Model-free RL directly interacts with the environment to improve the policy [1], but often suffers from poor sample efficiency, which is particularly important for embodied agents due to the high cost of real-world physical interaction.

Model-based RL methods such as DreamerV4 [11] and TD-MPC2 [33] instead learn an action-conditioned world model and use it for test-time search [23, 33] or model-based policy optimization [11, 34], making it more closely related to the Experience Machine setting. From the EM perspective, model-based RL can be viewed as a specific organization of the interaction among policy, world model, and outcome evaluation, while EM provides a more general abstraction that allows these components to interact through a shared Experience Tape and further extends the computation to experience from multiple sources. In this view, prior RL research provides valuable foundations for how an Experience Machine may use interaction feedback to continually reinforce and update its own policy and internal models.

EM is also closely related to Meta-RL [12], particularly recurrent Meta-RL such as RL^2 [35]. These methods can be viewed as restricted forms of EM, where experience mainly consists of state-action-reward interactions and recurrent hidden states serve as implicit experience beliefs [36]. EM generalizes this view by allowing heterogeneous experience from imagination, environmental interaction, and external communication to jointly shape the evolving computation.

Adaptive embodied reasoning in EM is also related to adaptive reasoning in Large Language Models (LLMs) [37], where reasoning effort scales with task difficulty. Recent LLMs often acquire such behavior through Reinforcement Learning with Verifiable Rewards (RLVR) [13], which favors sparse outcome supervision over noisy intermediate rewards [18]. For embodied agents, costly physical interaction makes such sparse feedback insufficient, motivating richer intermediate supervision from human critiques and revised intents through External Communication (EC).

4.4 LLM-based Experience Machine

Recent LLM-based agents provide a natural early instantiation of experience-driven computation. Systems such as ReAct [38], Reflexion [39], Generative Agents [40], MemGPT [41], and Voyager [14] have explored increasingly systematic ways to organize observation, reasoning, action, feedback, memory, and goal generation into persistent agent loops. Voyager [14], in particular, provides a concrete instantiation that closely resembles the EM formulation in a code-controlled Minecraft environment, combining automatic intent proposal, environmental feedback, self-verification, and persistent skill memory for lifelong exploration. From the EM perspective, these works demonstrate that LLMs can serve as effective computational engines for organizing and exploiting accumulated experience in agent systems. However, most existing approaches are still realized through externally orchestrated interaction loops; more end-to-end instantiations in which experience itself drives computation and learning [42], especially for agents that continuously interact with the physical world, remain largely unexplored.

5 Conclusion

We introduce the Experience Machine (EM), a conceptual computational framework for experience-driven intelligence. EM treats experience as a continuously evolving Experience Tape and unifies reasoning, prediction, action, feedback, and interaction through the same underlying computation. By allowing different sources to write different contents onto the tape, diverse capabilities such as embodied reasoning, self-exploration, reinforcement learning, and human-machine interaction can naturally emerge within a common framework. We hope EM provides a useful abstraction for experience-driven intelligence and serves as a foundation for future concrete instantiations.

References

[1] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine. Serl: A software suite for sample-efficient robotic reinforcement learning. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 16961–16969. IEEE, 2024.

[2] K. Lei, H. Li, D. Yu, Z. Wei, L. Guo, Z. Jiang, Z. Wang, S. Liang, and H. Xu. Rl-100: Performant robotic manipulation with real-world reinforcement learning. arXiv preprint arXiv:2510.14830, 2025.

[3] L. Floridi and M. Chiriatti. Gpt-3: Its nature, scope, limits, and consequences. Minds and machines, 30(4):681–694, 2020.

[4] G. Team. Gen-1.5: Embodied foundation models are one-shot learners. Generalist AI Blog, 2026. https://generalistai.com/blog/gen-1.5.

[5] Y. Jiang, Y. Chebotar, R. Zheng, F. Hu, Y. Ge, J. Wu, T. Dai, S. Reed, L. Fei-Fei, Y. Zhu, et al. Robottt: Context scaling for robot policies. arXiv preprint arXiv:2607.15275, 2026.

[6] A. Tandon, K. Dalal, X. Li, D. Koceja, M. Rød, S. Buchanan, X. Wang, J. Leskovec, S. Koyejo, T. Hashimoto, et al. End-to-end test-time training for long context. arXiv preprint arXiv:2512.23675, 2025.

[7] D. Ha and J. Schmidhuber. World models. arXiv preprint arXiv:1803.10122, 2(3):440, 2018.

[8] S. Gao, W. Liang, K. Zheng, A. Malik, S. Ye, S. Yu, W.-C. Tseng, Y. Dong, K. Mo, C.-H. Lin, et al. Dreamdojo: A generalist robot world model from large-scale human videos. arXiv preprint arXiv:2602.06949, 2026.

[9] A. Turing. Turing machine. Proc London Math Soc, 242:230–265, 1936.

[10] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch. Decision transformer: Reinforcement learning via sequence modeling. Advances in neural information processing systems, 34:15084–15097, 2021.

[11] D. Hafner, W. Yan, and T. Lillicrap. Training agents inside of scalable world models. arXiv preprint arXiv:2509.24527, 2025.

[12] J. Beck, R. Vuorio, E. Zheran Liu, Z. Xiong, L. Zintgraf, C. Finn, and S. Whiteson. A tutorial on meta-reinforcement learning. Foundations and Trends in Machine Learning, 18(2-3):224–384, 2025.

[13] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948, 2025.

[14] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023.

[15] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. pi0.5: a vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054, 2025.

[16] A. Liang, Y. Korkmaz, J. Zhang, M. Hwang, A. Anwar, S. Kaushik, A. Shah, A. S. Huang, L. Zettlemoyer, D. Fox, et al. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. arXiv preprint arXiv:2603.02115, 2026.

[17] O. Groth, M. Wulfmeier, G. Vezzani, V. Dasagi, T. Hertweck, R. Hafner, N. Heess, and M. Riedmiller. Is curiosity all you need? on the utility of emergent behaviours from curious exploration. arXiv preprint arXiv:2109.08603, 2021.

[18] D. Silver and R. S. Sutton. Welcome to the era of experience. Google AI, 1(11):6, 2025.

[19] M. Janner, Q. Li, and S. Levine. Offline reinforcement learning as one big sequence modeling problem. Advances in neural information processing systems, 34:1273–1286, 2021.

[20] S. Reed, K. Zolna, E. Parisotto, S. G. Colmenarejo, A. Novikov, G. Barth-Maron, M. Gimenez, Y. Sulsky, J. Kay, J. T. Springenberg, et al. A generalist agent. arXiv preprint arXiv:2205.06175, 2022.

[21] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, et al. World action models are zero-shot policies. arXiv preprint arXiv:2602.15922, 2026.

[22] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, F. Han, M. Yu, Z. Gao, N. Xue, X. Zhu, et al. Causal world modeling for robot control. arXiv preprint arXiv:2601.21998, 2026.

[23] H. Bi, Z. Zhou, Y. Tang, J. Pang, S. Huang, H. Liu, R. Wang, S. Huang, Y. Wang, Y. Cheng, R. Zhao, Z. Li, H. Tan, X. Liu, J. Wan, J. Liu, M. Zhao, F. Bao, and J. Zhu. Motus2: A self-evolving general world model for dexterous manipulation, 2026. URL https://arxiv.org/abs/2608.30237.

[24] Y. Wang, X. Li, P. Xie, P. Yang, B. Nie, Y. Cai, Q. Zhang, C. Qu, J. Wu, J. Song, et al. Learning while deploying: Fleet-scale reinforcement learning for generalist robot policies. arXiv preprint arXiv:2605.00416, 2026.

[25] M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, H. Wang, J. Tang, K. Stachowicz, et al. Mem: Multi-scale embodied memory for vision language action models. arXiv preprint arXiv:2603.03596, 2026.

[26] M. Koo, D. Choi, T. Kim, K. Lee, C. Kim, Y. Seo, and J. Shin. Hamlet: Switch your vision-language-action model into a history-aware policy. In International Conference on Learning Representations, volume 2026, pages 101537–101558, 2026.

[27] Q. Lian, K. Yu, and L. Zhang. Reflective vla: In-context action consequences make vlas generalize. arXiv preprint arXiv:2606.25215, 2026.

[28] S. Wang, J. Shi, S. Fei, Z. Fu, L. Ji, J. Gong, and X. Qiu. In-context world modeling for robotic control. arXiv preprint arXiv:2606.26025, 2026.

[29] M. Liu, D. Pathak, and A. Agarwal. Locoformer: Generalist locomotion via long-context adaptation. arXiv preprint arXiv:2509.23745, 2025.

[30] S. AI. Introducing s1: In-context learning for robotics. August 2026. URL https://skild.ai/blogs/s1.

[31] B. Wickramasinghe, G. Saha, and K. Roy. Continual learning: A review of techniques, challenges, and future directions. IEEE Transactions on Artificial Intelligence, 5(6):2526–2546, 2023.

[32] L. R. Squire, L. Genzel, J. T. Wixted, and R. G. Morris. Memory consolidation. Cold Spring Harbor perspectives in biology, 7(8):a021766, 2015.

[33] N. Hansen, H. Su, and X. Wang. Td-mpc2: Scalable, robust world models for continuous control. In International Conference on Learning Representations, volume 2024, pages 47376–47405, 2024.

[34] S. Wang, S. Liu, W. Ye, J. You, and Y. Gao. Efficientzero v2: Mastering discrete and continuous control with limited data. arXiv preprint arXiv:2403.00564, 2024.

[35] Y. Duan, J. Schulman, X. Chen, P. L. Bartlett, I. Sutskever, and P. Abbeel. Rl˜2: Fast reinforcement learning via slow reinforcement learning. arXiv preprint arXiv:1611.02779, 2016.

[36] J. X. Wang, Z. Kurth-Nelson, D. Tirumala, H. Soyer, J. Z. Leibo, R. Munos, C. Blundell, D. Kumaran, and M. Botvinick. Learning to reinforcement learn. arXiv preprint arXiv:1611.05763, 2016.

[37] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford, et al. Gpt-4o system card. arXiv preprint arXiv:2410.21276, 2024.

[38] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629, 2022.

[39] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. Advances in neural information processing systems, 36: 8634–8652, 2023.

[40] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In Proceedings of the 36th annual acm symposium on user interface software and technology, pages 1–22, 2023.

[41] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. Memgpt: Towards llms as operating systems. arXiv preprint arXiv:2310.08560, 2023.

[42] X. Luo, Y. Zhang, Z. He, Z. Wang, S. Zhao, D. Li, L. K. Qiu, and Y. Yang. Agent lightning: Train any ai agents with reinforcement learning. arXiv preprint arXiv:2508.03680, 2025.